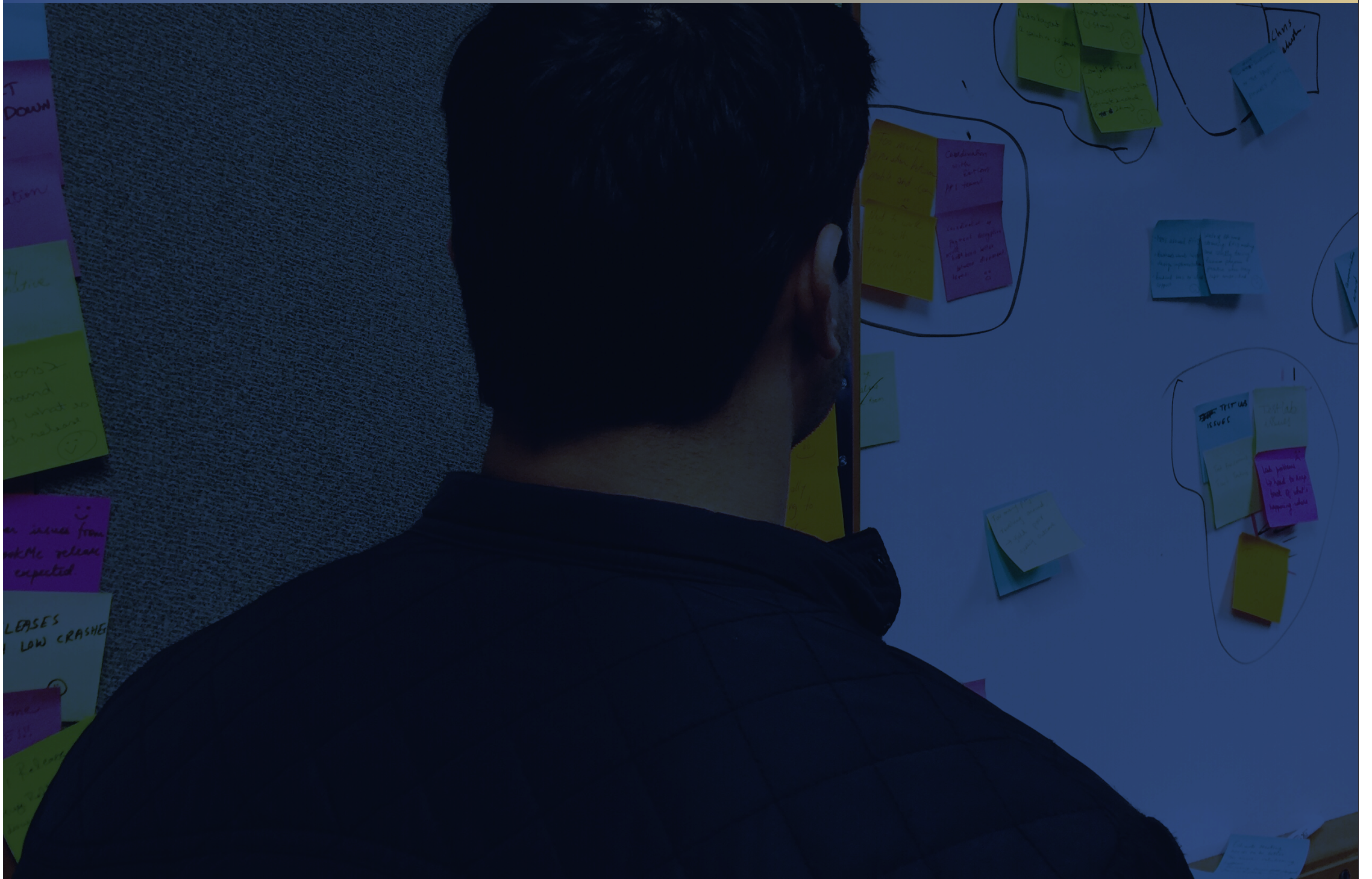




AULA 2

Abordagens para Engenharia e Especificação de Sistemas de Software Inteligentes



Objetivos de aprendizagem da aula

Ao final desta aula, você irá:

Que história é essa?

Nesta segunda aula, será fornecida uma visão geral a respeito de abordagens para a engenharia de sistemas de *software* inteligentes, apresentando e explicando conceitos como BizDev, DevOps/MLOps e experimentação contínua, que são comumente utilizados nesse contexto. Também serão discutidas boas práticas de engenharia de *software* recomendadas para abordagens de engenharia de sistemas de *software* inteligentes. Em seguida, abordaremos a definição do problema e requisitos, e veremos em detalhes como realizar a ideação e a especificação de sistemas de *software* inteligentes.

Engenharia de sistemas de *software* inteligentes

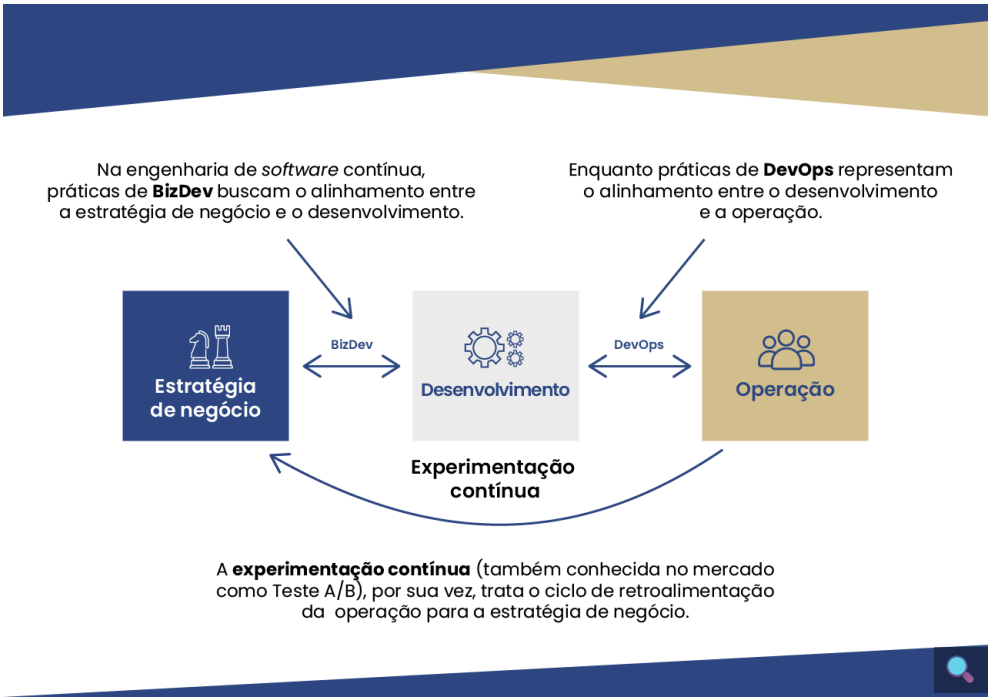
BizDev, DevOps e experimentação contínua

Os avanços na área de *machine learning* e na capacidade de processamento e a disponibilidade cada vez maior de dados têm tornado possível, em muitos casos, substituir funções cognitivas humanas e automatizar decisões de forma escalável com base em aprendizado automático obtido a partir de dados. Não é à toa que sistemas de *software* inteligentes, que incorporam componentes de *machine learning*, têm-se mostrado cada vez mais viáveis e considerados com frequência em contextos de inovação e transformação digital das empresas.

Fique ligado!

De forma resumida, a transformação digital pode ser vista como um processo em que empresas investigam o uso de tecnologias digitais para inovar sua forma de operação, buscando resolver problemas de negócio e alcançar objetivos estratégicos (KALINOWSKI *et al.*, 2020).

Nesse contexto, o alinhamento do desenvolvimento de soluções de *software* com a estratégia de negócio e com a operação tem sido o tema da engenharia de *software* contínua (FITZGERALD; STOL, 2017).



Para exemplificar, em contextos de transformação digital, normalmente se busca identificar um **produto mínimo viável** (ou *MVP – minimal viable product*) de *software* que permita testar hipóteses de negócio alinhadas com a estratégia de negócio da empresa. **Esse produto precisa ser desenvolvido refletindo esse alinhamento com a estratégia e ser posto em operação junto aos seus usuários finais para que as hipóteses de negócio possam ser de fato testadas.** O teste das hipóteses de negócio com o *software* em operação é a experimentação contínua, fechando um ciclo de retroalimentação. É essa avaliação que permitirá planejar incrementos (ou alternativas) para o produto para buscar melhorar cada vez mais os resultados de negócio. A busca pela melhoria resulta em novas hipóteses de negócio, a serem testadas no próximo ciclo de experimentação contínua.

Exemplo de abordagem para a engenharia de sistemas de *software* inteligentes: Lean R&D

Agora que estudamos esses conceitos, veja, no vídeo a seguir, um exemplo de uma abordagem ágil para a engenharia de sistemas de *software* inteligentes: a abordagem Lean R&D. A abordagem foi criada pelo professor desta disciplina em colaboração com outros professores da PUC-Rio e profissionais do mercado e tem sido usada na prática em iniciativas de transformação digital.

Assista o vídeo

Em sistemas de *software* inteligentes, que incorporam componentes de *machine learning*, não basta somente alinhar o desenvolvimento do *software* com a operação (DevOps), mas também manter os modelos de *machine learning* atualizados e representativos para a operação a todo instante. Há diversas situações em que fenômenos que afetam o comportamento dos dados fazem com que as previsões dos modelos deixem de fazer sentido.



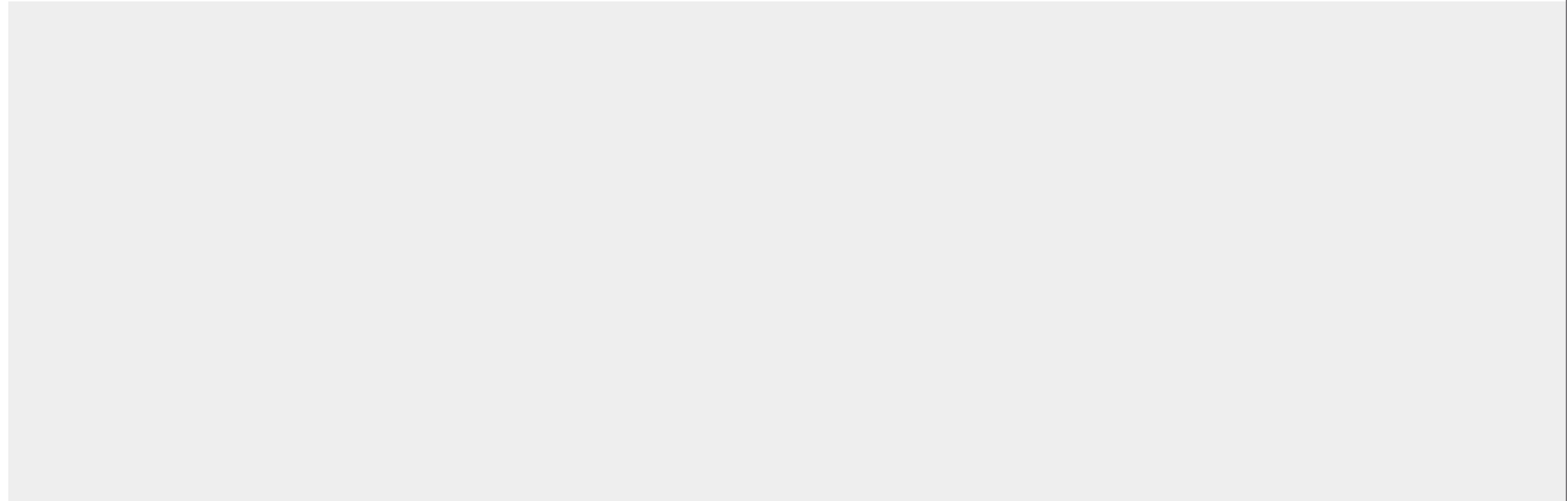
Durante a pandemia, por exemplo, as pessoas mudaram seus padrões de consumo. Em relação ao vestuário, em geral, passaram a comprar menos roupas sociais e mais roupas casuais. Isso indica uma mudança de comportamento que um modelo treinado antes da pandemia não teria como prever.

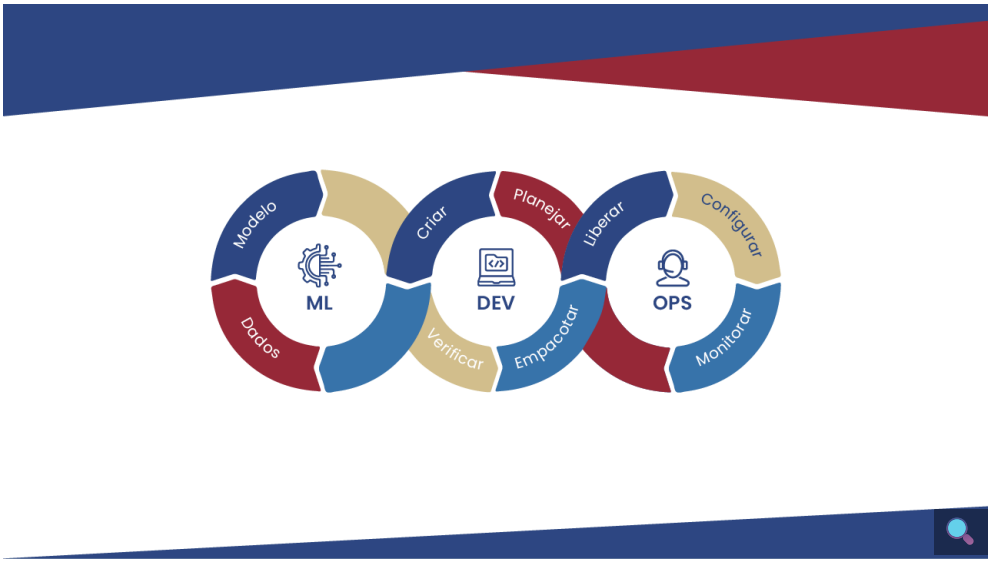
Existe a necessidade de identificar mudanças o quanto antes e de retreinar e reimplantar uma nova versão do modelo, mesmo que outras partes do sistema não sejam alteradas.

Aprenda mais

MLOps é o alinhamento da construção dos modelos de *machine learning* com o desenvolvimento de *software* e a operação. Assim, contempla as melhores práticas para colaboração entre cientistas de dados, desenvolvedores e profissionais de operação.

A figura a seguir ilustra a necessidade de se alinhar os modelos com o desenvolvimento e com a operação.





Muitas vezes não se pode simplesmente implantar um modelo de *machine learning* treinado *off-line* como um serviço de previsão definitivo. **Em diversas situações, será necessário um *pipeline* de várias etapas para retreinar e implantar automaticamente um modelo, de forma contínua. Esse *pipeline* adiciona complexidade porque precisa automatizar etapas que os cientistas de dados normalmente executam manualmente antes da implantação para treinar e validar novos modelos.**

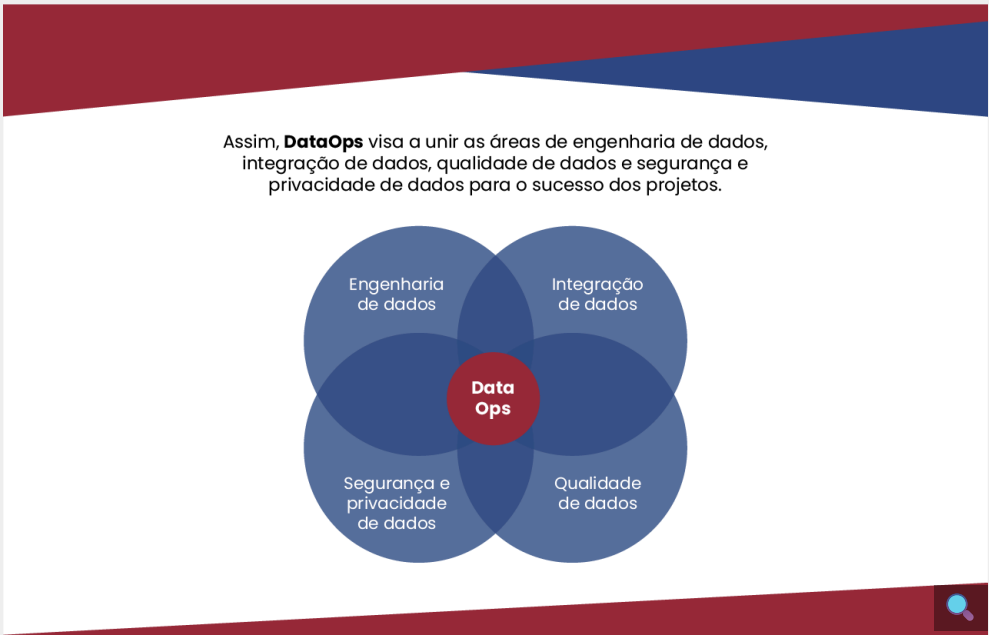
As fases típicas de um *pipeline* de MLOps contemplam: coleta e análise de dados, preparação dos dados, treinamento do modelo, validação do modelo, colocar o modelo em operação, monitorar o modelo e retreiná-lo sempre que necessário.

Normalmente o que dispara a necessidade de atualização do modelo é a identificação de um fenômeno conhecido como ***concept drift*** durante a monitoração do modelo.

Fique ligado!

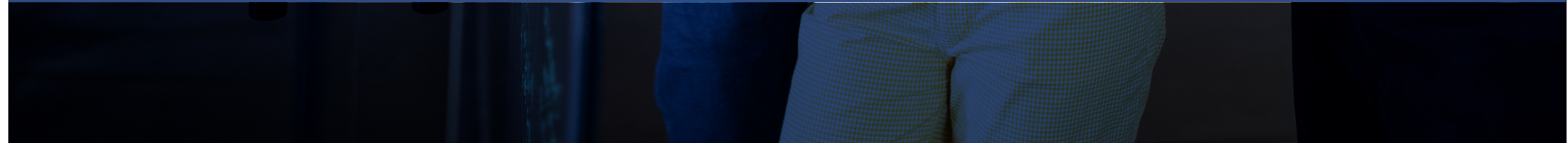
Dizemos que ocorreu um ***concept drift*** quando mudanças no comportamento dos dados são identificadas. Assim, é comum que na etapa de monitoração do modelo haja mecanismos de identificação de ***concept drift***, a fim de disparar o *pipeline* de MLOps para, por exemplo, iniciar o retreinamento do modelo com novos dados.

Ainda em relação aos dados, **DataOps** envolve zelar pela qualidade dos dados em todos os aspectos e fornecê-los de forma confiável para uso, seja na operação, seja para atividades como a construção de modelos.



Requisitos de sistemas de *software* inteligentes

Conceitos básicos da engenharia de requisitos





“A parte mais difícil da construção de um sistema de *software* é decidir precisamente o que deve ser construído. Nenhuma outra parte do trabalho conceitual é tão difícil quanto estabelecer detalhadamente os requisitos técnicos, incluindo todas as interfaces com pessoas, máquinas e outros sistemas de *software*.”

Frederick P. Brooks Jr. (1987)

A frase clássica acima, amplamente conhecida na área de engenharia de *software*, foi publicada em uma edição da revista *IEEE Computer* de 1987, mas seu conteúdo é consenso até hoje. Na verdade, quando se trata de requisitos de sistemas de *software* inteligentes, a dificuldade reportada ainda aumenta. Ao longo deste conteúdo, iremos abordar a especificação de sistemas de *software* inteligentes em detalhes e tentar simplificar o processo. Iniciaremos com algumas definições-chave da área de requisitos.

De acordo com o International Requirements Engineering Board (IREB) (GLINZ *et al.*, 2020), que é o principal órgão de certificação profissional na área de requisitos, o termo “**requisito**” denota três conceitos:

- uma necessidade percebida por um interessado;
- uma capacidade ou propriedade que um sistema deve ter;
- uma representação documentada de uma necessidade, capacidade ou propriedade.

Ou seja, não basta especificar o que o sistema fará; é preciso também descrever capacidades ou propriedades relacionadas a outras características do *software*, como compatibilidade, confiabilidade, desempenho, manutenibilidade, portabilidade, segurança e usabilidade.

É importante ressaltar que há diferentes visões para requisitos. **Os requisitos do usuário expressam resultados desejados para superar problemas no mundo real.** Normalmente, eles são especificados de forma mais abstrata, focados no problema e sem prover muitos detalhes do que exatamente será construído. Assim, eles não costumam fornecer detalhes sobre telas e regras de negócio mais específicas.

Os requisitos do sistema, por sua vez, focam na especificação das soluções a serem desenvolvidas utilizando sistemas novos e existentes. Eles devem fornecer detalhes sobre **o que** será construído, incluindo a especificação das interfaces e da interação com pessoas, máquinas e outros sistemas de *software*. Não devem, entretanto, entrar em detalhes sobre **como** o sistema será construído, já que isso será especificado por equipes técnicas durante a arquitetura e o projeto do sistema. Para que um projeto seja considerado bem-sucedido, os requisitos do sistema devem satisfazer os requisitos do

usuário, resolvendo o problema que ele enfrenta. Essa relação está representada na figura a seguir.



Em abordagens mais tradicionais, os requisitos do usuário são comumente especificados em documentos conhecidos como documento de visão ou documento de requisitos do usuário. Já os requisitos do sistema são comumente especificados em documentos conhecidos como documento de requisitos do sistema, ou do *software*. Veremos mais adiante como isso é tratado em abordagens ágeis, que são hoje as mais utilizadas pelo mercado. Antes disso, precisamos ainda entender o conceito de engenharia de requisitos.

A **engenharia de requisitos** pode ser definida como a abordagem sistemática e disciplinada para a **especificação e o gerenciamento de requisitos** com o objetivo de compreender os desejos e as necessidades das partes interessadas e minimizar o risco de entregar um sistema que não atenda a esses desejos e necessidades (GLINZ *et al.*, 2020). Ou seja, é a disciplina da engenharia de *software* que compreende produzir e gerenciar os requisitos.

As atividades típicas relacionadas com a produção e a gerência dos requisitos estão ilustradas na figura a seguir. Tipicamente, a produção envolve o levantamento, a especificação e a verificação e validação de requisitos. Já a gerência de requisitos envolve gerência de configuração (versionamento), controle de mudanças, rastreabilidade (manter informações sobre quem pediu determinado requisito e quais partes do sistema ele afeta) e gerência da qualidade dos requisitos.



Tipos de requisitos

Requisitos podem ser divididos nos seguintes três tipos: **requisitos funcionais**, **requisitos não funcionais** e **regras de negócio** (SOMMERVILLE, 2019). Cabe ressaltar que, em algumas fontes, os requisitos não funcionais são ainda subdivididos em requisitos de qualidade e restrições (GLINZ *et al.*, 2020). Vamos entender cada um desses requisitos a seguir.



Os requisitos funcionais descrevem **as funcionalidades que o software deve ser capaz de realizar**. Ou seja, são requisitos diretamente ligados à funcionalidade do *software*. É comum que sejam inicialmente descritos de forma abstrata (como requisitos funcionais do usuário) e depois detalhados (como requisitos funcionais do sistema).

Clique nas opções 'Avançar' e 'Voltar' para ver exemplos.



Interativo

Descrição do interativo

Os requisitos não funcionais, em vez de informarem o que o sistema fará, descrevem **características de qualidade ou restrições**. De acordo com o modelo de qualidade de produtos de *software* ISO 25010, as características de qualidade que podem precisar ser especificadas, além dos aspectos funcionais em si capturados pelos requisitos funcionais, são: compatibilidade, confiabilidade, desempenho, manutenibilidade, portabilidade, segurança e usabilidade. As restrições, por sua vez, podem ser relacionadas a escolhas de tecnologia da organização, como, por exemplo, restringir as possibilidades ao uso de um sistema de gerenciamento de banco de dados específico.

Exemplos de requisitos não funcionais:

>

Exemplos:

[RNF1] A predição realizada pelo sistema deve ser obtida dentro de três segundos.
(requisito não funcional de desempenho)

[RNF2] Deve ser possível executar o sistema nas plataformas mobile Android e iOS.
(requisito não funcional de desempenho)

É importante ressaltar que os requisitos não funcionais podem ser mais críticos que os requisitos funcionais e que devem ser descritos de forma mensurável e verificável. **Em muitas situações, o sistema que não satisfaz requisitos não funcionais se torna um sistema inútil.** Basta pensar em um sistema de tempo real que não atende às

restrições de desempenho necessárias para o seu uso ou em um sistema voltado para usuários com deficiência sem tratar aspectos da característica de usabilidade referentes à acessibilidade.



Por fim, os requisitos de domínio descrevem **regras de negócio que devem ser atendidas pelo sistema**. Normalmente, essas regras ajudam a detalhar os requisitos funcionais com aspectos que precisarão ser considerados ou obedecidos em computações específicas. Se os requisitos de domínio não forem satisfeitos, **o sistema pode se tornar não prático**.

Exemplos de requisitos de domínio (regras de negócio):

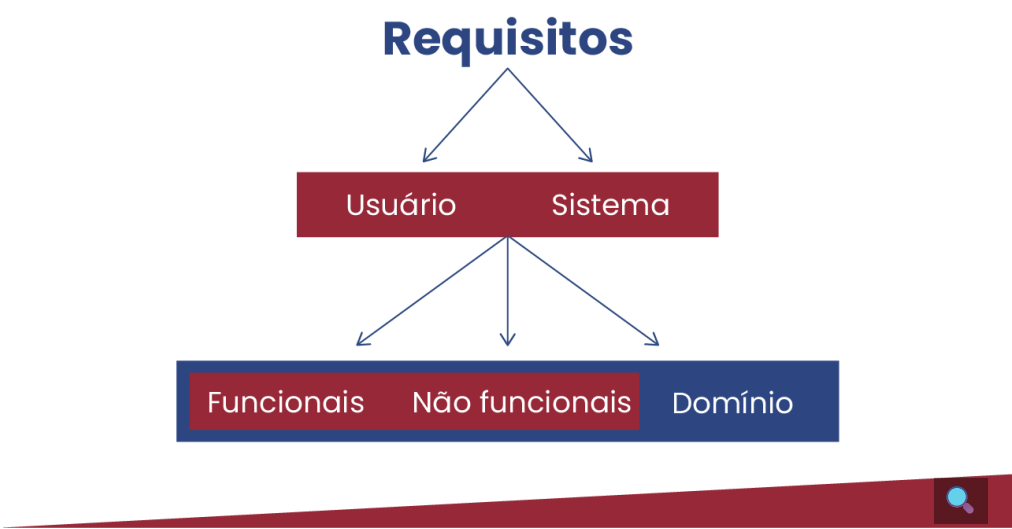
> Exemplos:

[RN1] Não podem ser concedidos empréstimos para pessoas físicas que não tenham comprovante de rendimentos.

[RN2] Um cliente de um empréstimo é considerado inadimplente caso o atraso no pagamento de alguma de suas parcelas tenha ultrapassado 30 dias.

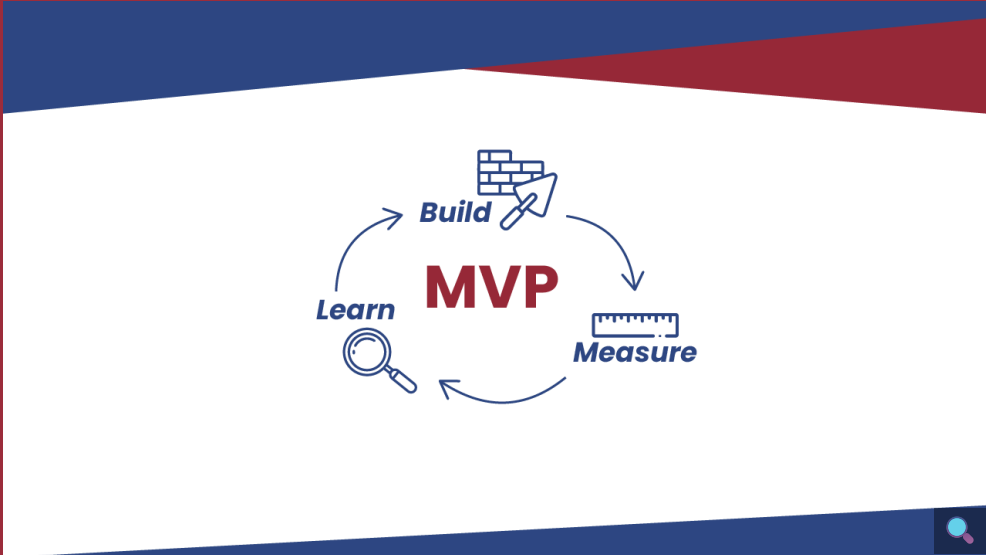
Note que nesses exemplos de regras de negócio, a palavra “**sistema**” não aparece. Isso é comum, porque regras de negócio costumam valer dentro de um domínio (área de negócio), de forma independente do sistema a ser construído. Entretanto, é importante que aquelas regras que o sistema terá que satisfazer ou que afetem sua construção de alguma forma sejam especificadas.

Podemos sumarizar a discussão sobre tipos de requisitos com base na figura a seguir.



É comum que requisitos sejam inicialmente descritos como requisitos do usuário, menos detalhados e mais voltados para os objetivos do usuário, e posteriormente detalhados como requisitos do sistema. Independentemente do nível de detalhamento, eles podem ser requisitos funcionais ou requisitos não funcionais do sistema em si ou requisitos de domínio da área para a qual o sistema está sendo construído.

Um conceito comumente utilizado em contextos de transformação digital é o do **MVP**. Ele se popularizou com a abordagem *lean startup*, que tem como princípio fundamental o uso do ciclo de aprendizado evolutivo *build-measure-learn* (crie-meça-aprenda) para transformar ideias em produtos, medir como os clientes respondem a esses produtos e, então, avaliar se a ideia deve ser levada adiante.



Aprenda Mais

Um MVP pode ser definido como uma versão enxuta de um novo produto, que permite à equipe coletar o máximo de aprendizado validado com o menor esforço. Um MVP precisa permitir avaliar hipóteses de negócio, como, por exemplo, avaliar se o uso de um recurso de inteligência computacional permitirá aumentar as vendas da empresa em X%. No caso de produtos de *software*, os MVPs devem ser construídos de forma enxuta, mas pensando na sua possível evolução por meio de incrementos futuros. Na área de *software*, sabemos do elevado custo de evoluir soluções que não foram construídas com qualidade e que construir um MVP com qualidade em mente pode evitar grandes perdas com retrabalho futuro. Ou seja, **um MVP de *software* deve ser mínimo em escopo, não em qualidade.**

Nos contextos de transformação digital, a elaboração do documento de visão é comumente substituída por dinâmicas de ideação e a geração de um **MVP Canvas**. Além disso, em contextos ágeis de desenvolvimento, como o que vimos na última aula, hoje predominantes no mercado, o documento de requisitos do sistema é comumente substituído por um ***backlog do produto* (product backlog)**, em que os itens são **histórias de usuário** (*user stories*).

Em relação à especificação da funcionalidade em si, o MVP Canvas lista *features* identificadas durante as dinâmicas de ideação. ***Features* são funcionalidades ou recursos que têm como propósito adicionar uma nova entrega de valor e experiência para seus usuários.** Ou seja, o termo *feature* é utilizado de maneira similar ao conceito de um requisito funcional do usuário, descrevendo de forma abstrata uma funcionalidade que o *software* deve prover. Essas *features* são, então, detalhadas em histórias de usuário, normalmente pelo PO. Essas histórias de usuário, por sua vez, serão quebradas em tarefas pela equipe de desenvolvimento.

23/02/2026, 21:19

Abordagens para Engenharia e Especificação de Sistemas de Software Inteligentes

A figura a seguir mostra a estrutura desses agrupamentos:

Um **MVP** tem *features* que são detalhadas em **histórias de usuário**, que, por sua vez, são detalhadas em **tarefas**.

MVP

Feature

História do usuário

Tarefa

A dinâmica de ideação é o ponto de partida, e existem hoje diversas abordagens para conduzir essa dinâmica:

- *design thinking*;
- *design sprints*;
- *lean startup*;
- *lean inception*.

Exemplificaremos essas dinâmicas por meio da *lean inception*, definida como a combinação do *design thinking* com *lean startup* para decidir o produto mínimo viável (MVP). Mais informações sobre essa dinâmica podem ser encontradas no livro escrito pelo criador da dinâmica, Paulo Caroli (2018).

A figura a seguir ilustra a agenda típica de uma *lean inception*. É possível observar uma sequência de atividades sendo conduzidas em sessões com a participação dos diferentes interessados. Para cada uma dessas atividades, há *templates* gratuitos disponíveis, e elas estão descritas resumidamente a seguir.

Agenda de um *lean inception*

Clique nos itens para acessar as explicações

Seg

Ter

Introdução & kick off

lança

Interativo

Descrição do interativo

https://pucrio.grupoa.education/plataforma/course/2841713/content/64848312

13/20

React de Mercado

Caso queira saber mais sobre o método *Lean Inception*, te convidamos para ver mais sobre o tema, explicado pelo próprio criador do método, Paulo Caroli, para entender melhor sobre o seu propósito, quando aplicar, quem deve estar envolvido, pontos de atenção e recomendações.



Vejamos o resultado fictício de um MVP Canvas seguindo o *template* da *lean inception* na figura a seguir, referente a um banco que busca reduzir a inadimplência dos seus clientes no pagamento dos seus empréstimos.

Canvas MVP

Personas segmentadas Gerente do banco Ciente	Proposta do MVP Validar se é possível reduzir inadimplência com o uso de inteligência computacional	Resultado esperado Reduzir o número de inadimplentes três meses após a concessão do empréstimo de 20% para 10%
	Funcionalidades Classificação automática na concessão Renegociar empréstimo Acompanhar situação do cliente	Aumentar o número de empréstimos concedidos nos últimos 30 dias em 10%
Jornadas Conceder empréstimo Renegociar empréstimo Acompanhar situação do cliente	Custo e cronograma 1 <i>squad</i> de 3 desenvolvedores e 1 cientista de dados por 4 meses + <i>Scrum master</i> + PO + analista de infra/DevOps	Métricas para validar as hipóteses do negócio Número de inadimplentes três meses após a concessão do empréstimo Número de empréstimos concedidos nos últimos 30 dias

Além disso, histórias de usuário são comumente complementadas com descrições informais que envolvem a listagens de campos, regras de negócio que devem ser obedecidas, regras de validação (como campos obrigatórios, entre outros), protótipos de telas (*wireframe*, sem implementação) e aspectos não funcionais a serem considerados (como segurança ou usabilidade).



Outra prática muito comum em métodos ágeis é complementar a história de usuário com **critérios de aceitação** (também conhecidos como cenários BDD – *behavior-driven development*). No contexto ágil, o *definition of ready* que vimos na última aula (que significa que uma história está preparada para ser implementada) é comumente associado a ter detalhado a história de usuário e os seus critérios de aceitação. Já o *definition of done* (que significa que uma história foi implementada e está feita) é obtido quando uma história de usuário foi implementada e testada, atendendo aos critérios de aceitação.

Um **critério de aceitação** de uma história de usuário descreve um conjunto ordenado de comportamentos baseado em uma entrada para alcançar um resultado específico de uma funcionalidade. Costuma ser composto de descrições adicionais utilizadas para a aceitação da história de usuário. O formato padrão para os critérios de aceitação é o “**Dado que** <contexto?> **quando** <ação?> **então** <resultado?>”, em que:

- <contexto?> descreve em qual ponto o cenário irá iniciar.
- <ação?> define a ação (ou evento) que está sendo realizada pelo usuário no contexto descrito.
- <resultado?> define o resultado esperado da ação.

Veja um exemplo para a história de usuário US01 do exemplo anterior, referente à classificação automática de clientes em bons ou maus pagadores utilizando *machine learning*.

História de usuário:

[US01] Como gerente do banco, **quero** realizar a classificação automática de um cliente **para** saber se ele irá pagar um empréstimo e decidir sobre a sua concessão.

Critérios de aceitação:

- **Dado que** um bom pagador solicita um empréstimo
- **Quando** consulto a classificação automática
- **Então** o sistema me sugere **conceder** o empréstimo em pelo menos 90% dos casos.
- **Dado que** um mau pagador solicita um empréstimo
- **Quando** consulto a classificação automática
- **Então** o sistema me sugere **não conceder** o empréstimo em pelo menos 90% dos casos.

Considerando esses cenários de aceitação, o sistema precisaria ser testado utilizando dados de bons e maus pagadores não vistos durante a construção do modelo. Para a avaliação, seria utilizada uma métrica muito conhecida no meio de algoritmos de classificação, que é o *recall* (percentual corretamente recuperado), para bons e para maus pagadores. Se o *recall* para bons pagadores for acima de 90%, o sistema atenderá ao primeiro critério de aceitação. Se o *recall* para maus pagadores também for acima de 90%, o sistema atenderá aos dois critérios de aceitação. As histórias de usuário (com seu detalhamento) ficam armazenadas no *backlog* do produto, que deve ser mantido e refinado pelo *product owner*, conforme vimos na última aula.



Requisitos de sistemas de *software* inteligentes



Como explicado anteriormente, **a especificação baseada em histórias de usuário precisará ser complementada para subsidiar bem o trabalho do cientista de dados**. De fato, a especificação precisa é considerada a atividade mais difícil do desenvolvimento de sistemas baseados em *machine learning*.

Para subsidiar a construção do modelo de *machine learning* e a sua integração com o sistema, a história de usuário precisaria ser complementada com informações sobre o tipo de problema de *machine learning* sendo tratado (por exemplo, classificação, regressão, agrupamento ou associação), as informações de entrada disponíveis para o modelo, as saídas esperadas, as fontes de dados para o aprendizado, detalhes sobre a coleta dos dados, informações sobre a dinâmica de criação e atualização do modelo, informações sobre a dinâmica de execução do modelo, informações sobre como o modelo será avaliado antes de ser implantado (avaliação *off-line*),

como o modelo será monitorado e o que deverá ser monitorado, além de outras características de qualidade desejadas (qualidade dos dados, explicabilidade, aspectos éticos, confiabilidade/acurácia do modelo, entre outros) e o grau em que se deseja cada uma. Além disso, ainda é preciso pensar nos requisitos não funcionais e de infraestrutura referentes ao sistema em que o modelo será incluído. Ou seja, especificar sistemas baseados em *machine learning* é complexo.



Vejamos exemplos de problemas de classificação, regressão, agrupamento e associação no formato de histórias de usuário.

• Classificação

Feature:

[F01] Classificação automática na concessão de bons e maus pagadores

Histórias de usuário:
[US01] **Como** gerente do banco, **quero** realizar a classificação automática de um cliente **para** saber se ele irá pagar um empréstimo e decidir sobre a sua concessão .

• Regressão

Feature:

[F01] Predição do faturamento

Histórias de usuário:
[US01] **Como** diretor financeiro da empresa, **quero** saber o faturamento esperado **para** o próximo trimestre para apoiar o planejamento de investimentos.

• Agrupamento

Feature:

[F01] Identificação de Localidade para novas filiais

Histórias de usuário:
[US01] **Como** diretor de expansão, **quero** saber qual o local mais promissor para abrir uma nova filial **para** decidir sobre o local de abertura.

• Associação

Feature:

[F01] Oferta de novos produtos

Histórias de usuário:
[US01] **Como** cliente da loja X, **quero** ver produtos relacionados aos meus interesses **para** encontrar o produto ideal para minha necessidade.



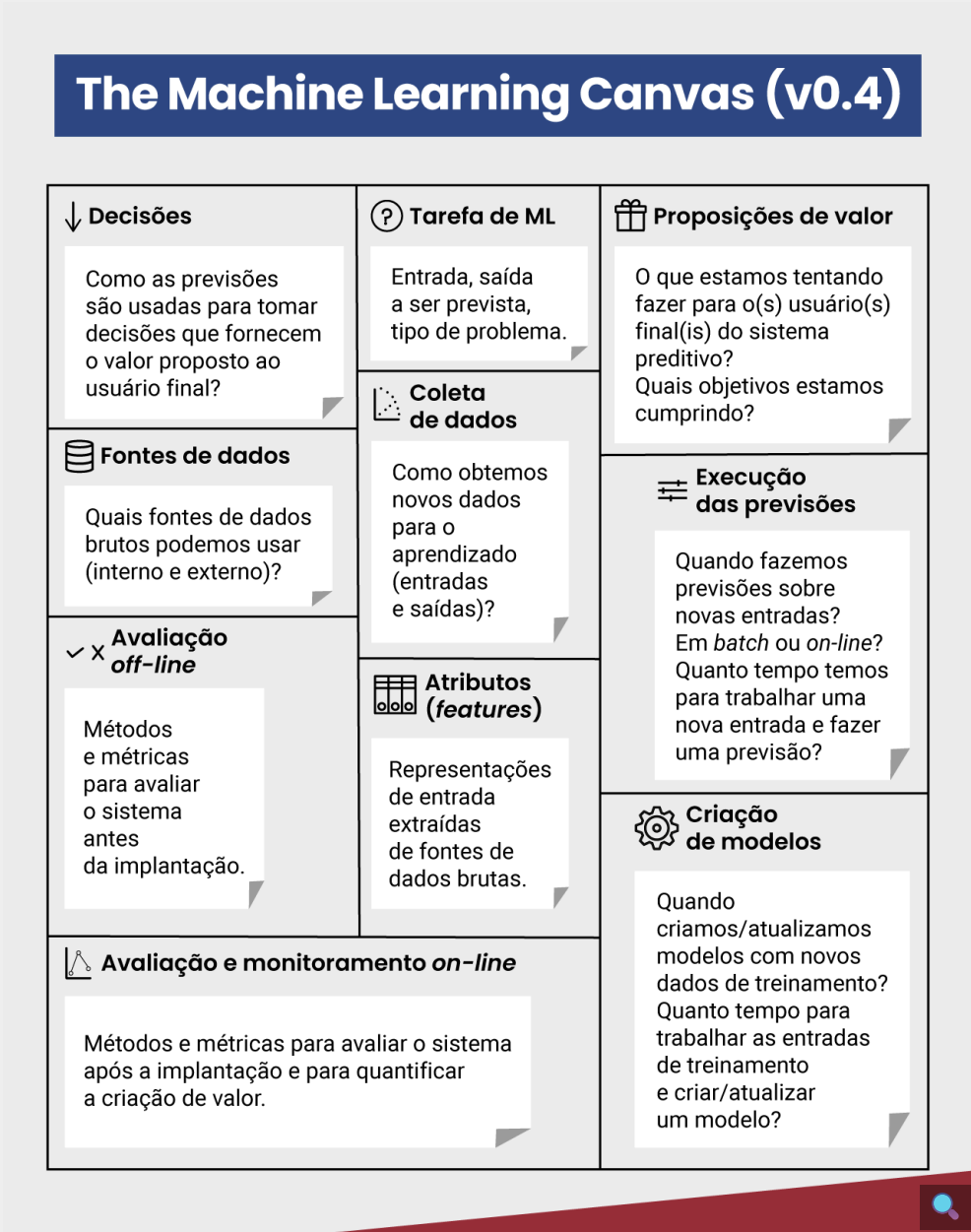
Independentemente do tipo de problema de *machine learning*, o formato de uma história de usuário é claramente insuficiente para subsidiar a construção do modelo, e a especificação precisa ser **complementada**. Note que normalmente uma *feature* de *machine learning* é mapeada em uma única história de usuário e que esta não provê nenhum detalhe para o cientista de dados.

ML Canvas

Uma alternativa para especificar aspectos relacionados a uma solução de *machine learning* é o **Machine Learning Canvas** (ou **ML**

Canvas), proposto por Dorard (2015). O **Machine Learning Canvas** descreve como sistemas de *machine learning* irão tornar predições em valor para os usuários finais, considerando elementos como a coleta de dados, a monitoração do modelo e a proposição de valor. Esta é provavelmente a abordagem mais amplamente utilizada hoje na prática para descrever sistemas que têm componentes de *machine learning*, embora também tenha claras limitações do ponto de vista de engenharia de *software*.

O ML Canvas pode ser visto na figura a seguir. Por um lado, ele é prático e captura informações essenciais para apoiar a construção ágil de algum modelo de *machine learning*. Por outro lado, ele é desintegrado do processo de desenvolvimento e claramente incompleto do ponto de vista de engenharia de *software*, não capturando perspectivas e preocupações que podem ser muito relevantes do ponto de vista prático. Por exemplo, ele não captura preocupações referentes à infraestrutura em que o modelo será servido, à forma de interação com o usuário, à forma de visualização dos dados, ao tamanho do modelo, à explicabilidade, à privacidade, entre muitas outras.



PerSpecML: especificação baseada em perspectivas

A abordagem de especificação de *machine learning* baseada em perspectiva (**PerSpecML – *Perspective-based specification of machine learning-enabled systems***) foi proposta por (VILLAMIZAR *et al.*, 2022) como parte de uma pesquisa de doutorado realizada no Departamento de Informática da PUC-Rio e é detalhada didaticamente em (KALINOWSKI *et al.*, 2023). Ela se diferencia das demais por introduzir um conjunto de perspectivas e preocupações relevantes para a especificação de sistemas baseados em *machine learning*. A abordagem foi avaliada junto a diversas empresas e mostrou vantagens significativas em relação à simples aplicação do ML Canvas, podendo ser considerada o estado da arte para a especificação de sistemas baseados em *machine learning* (KALINOWSKI *et al.*, 2023).



As perspectivas e preocupações foram levantadas considerando preceitos da engenharia de *software* e um conjunto de avaliações. Elas servem como uma espécie de *checklist*, assegurando que a especificação considerou os aspectos mais importantes para subsidiar apropriadamente a construção de sistemas baseados em *machine learning*.

A PerSpecML considera 5 perspectivas e 51 preocupações:

- objetivos de *machine learning* (7 preocupações a serem consideradas);
- experiência do usuário (7 preocupações a serem consideradas);
- infraestrutura (10 preocupações a serem consideradas);
- modelo (11 preocupações a serem consideradas);
- dados (16 preocupações a serem consideradas).



Interativo

Descrição do interativo

Reflexão

Agora que você sabe tudo sobre a especificação de sistemas de *software* inteligentes, chegou a hora de refletir. Com base na sua experiência e no conteúdo desta aula, reflita sobre a importância de realizar uma boa especificação de requisitos para sistemas inteligentes. Pense no impacto de deixar de tratar antecipadamente ao menos uma das preocupações de especificação exemplificadas no material.

Referências

Clique aqui para acessar as referências e créditos desta aula.

